```
import pandas as pd
```

INTRO TO PYTHON · SESSION 1

Fundamentals of Pandas

Series, DataFrames, indexing and a first look at real data with plots

[Instructor name] · [Date]

TODAY

Five steps from an empty notebook to a plot

01	**Setup**	Load the libraries and get the notebook ready
02	**Series**	One column of values, each with a label
03	**DataFrame**	Build a table, load a CSV file and explore it
04	**Indexing**	Pick out columns, rows and single cells
05	**Visualization**	Line, bar and pie charts, then plots of real app data

01 · SETUP

Four libraries, imported once at the top

```
import pandas, numpy, matplotlib
import seaborn
from matplotlib import pyplot
%matplotlib inline
```

The last line makes plots appear right below the cell. Missing seaborn? Run !pip install seaborn, restart the kernel, and import again.

Short names you will see elsewhere

Library	Common alias
pandas	pd
numpy	np
pyplot	plt
seaborn	sns

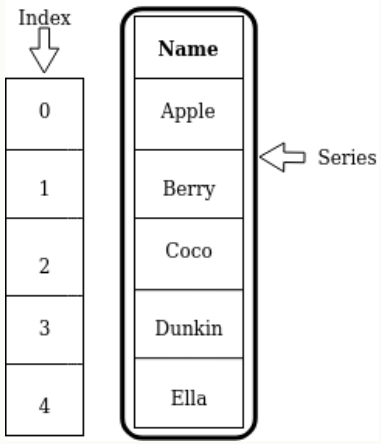
The notebook writes the full names, like pandas.Series. So do these slides.

02 · SERIES

Pandas stores data in two shapes

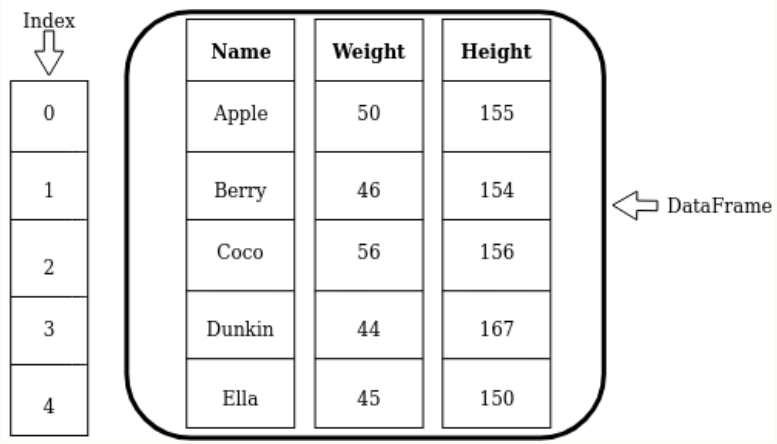
Series

A 1-D column of values. Each value has a **label**; all share one **data type**.



DataFrame

A 2-D table: **row labels**, **column names**, and a data type per column.



02 · SERIES

Creating a Series from a list

```
CPU_USAGE = pandas.Series(
[0.16, 0.07, 0.23, 0.24, 0.14,
4.99, 0.23, 0.47, 0.46, 0.17])

print(CPU_USAGE)
print(len(CPU_USAGE)) # 10
```

Ten CPU-load measurements. With no labels given, pandas numbers them **0, 1, 2 …** like a Python list.

What print shows

label	value
0	0.16
1	0.07
2	0.23
3	0.24
4	0.14

label	value
5	4.99
6	0.23
7	0.47
8	0.46
9	0.17

dtype: float64

02 · SERIES

Square brackets look up a label, not a position

```
MEM_USAGE = pandas.Series(
[0.16, 0.07, 0.23, 0.24, 0.14],
index=[3, 1, 4, 2, 7])

MEM_USAGE[3] # 0.16
MEM_USAGE[[1, 3]] # 0.07, 0.16
MEM_USAGE[[True, False,
False, True, True]] # labels 3, 2, 7
```

label 3	label 1	label 4	label 2	label 7
0.16	0.07	0.23	0.24	0.14

MEM_USAGE[3] is 0.16, the value **labeled** 3. It happens to sit first, in position 0.

A list of labels returns several values. A list of True/False, one per element, keeps only the True ones.

02 · SERIES

Change values with .loc[]

```
MEM_USAGE.loc[7] = 0.33 # step 1
MEM_USAGE.loc[[1, 3]] = [4, 2] # step 2
MEM_USAGE.loc[[True, False, False, True, True]] = [1.0, 2.0, 3.0] # step 3
```

label	3	1	4	2	7
start	0.16	0.07	0.23	0.24	0.14
step 1	0.16	0.07	0.23	0.24	0.33
step 2	2.00	4.00	0.23	0.24	0.33
step 3	1.00	4.00	0.23	2.00	3.00

Amber cells changed in that step. In step 3 the True values pick labels 3, 2 and 7, and the three new values go to them in order.

02 · SERIES · WATCH OUT

The = sign does not make a copy

Two names, one object

```
COPY_USAGE = MEM_USAGE
COPY_USAGE[3] = 1.23

COPY_USAGE[3] # 1.23
MEM_USAGE[3] # 1.23 too!
```

Changing the "copy" changed the original.

A real, independent copy

```
COPY_USAGE = MEM_USAGE.copy()
COPY_USAGE[3] = 1.23

COPY_USAGE[3] # 1.23
MEM_USAGE[3] # 0.16, unchanged
```

Use .copy() whenever you want to experiment safely.

03 · DATAFRAME

Two ways to get the same table

```
# 1. From a nested list
source_table = [['Apple', 50, 155],
['Berry', 46, 154], ['Coco', 56, 156],
['Dunkin', 44, 167], ['Ella', 45, 150]]
avatars = pandas.DataFrame(source_table,
columns=['Name', 'Weight', 'Height'])

# 2. From a CSV file
avatars_read = pandas.read_csv("avatars.csv")
```

read_csv takes the column names from the file's first line and works out each column's type.

Either way: avatars

	Name	Weight	Height
0	Apple	50	155
1	Berry	46	154
2	Coco	56	156
3	Dunkin	44	167
4	Ella	45	150

Row labels default to 0, 1, 2 … unless you pass index=.

03 · DATAFRAME

First questions to ask any new table

You type	On avatars you get	It answers
len(avatars)	5	How many rows?
avatars.shape	(5, 3)	How many rows and columns?
avatars.columns	Name, Weight, Height	What are the columns called?
avatars.dtypes	text, int64, int64	What kind of data is in each?
avatars.head(n)	first n rows (5 by default)	What does the data look like?
avatars.describe()	count, mean, min, max …	What are typical values?
avatars.info()	all of the above at once	Is anything missing?

No parentheses = an **attribute** you read. Parentheses = a **method** you call. Text columns show as object in older pandas and str in pandas 3.

03 · DATAFRAME

Real data: how two phone apps use CPU and memory

```
df_mystery = pandas.read_csv("sherlock/sherlock_mystery.csv")
```

200

rows: snapshots of a running app

14

columns: CPU_USAGE, num_threads, vsize …

2

apps: Facebook (121 rows) and WhatsApp (79)

Quiz Run **df_mystery.describe()**. What is the mean of num_threads? 71.365 · 62.000 · 0.075 · none of these

04 · INDEXING

[] picks columns, .loc[] picks rows by label

You type (df6 = first 6 rows)	You select	You get
df6['ApplicationName']	one column	Series
df6[['ApplicationName', 'CPU_USAGE']]	several columns	DataFrame
df6[[True, False, False, True, True, False]]	rows 0, 3, 4	DataFrame
df6.loc[1]	row 1	Series
df6.loc[[1, 3, 5]]	rows 1, 3, 5	DataFrame
df6.loc[1, 'num_threads']	one cell	47
df6.loc[[1, 2, 5], ['num_threads', 'CPU_USAGE']]	rows × columns	DataFrame

One label in → a Series or a single value. A **list** of labels in → a DataFrame. To **change** cells, always use .loc: df6.loc[2, 'CPU_USAGE'] = 0.01

04 · INDEXING

Slices with .loc include both ends

You type	You select
df6.loc[1, :]	row 1, every column
df6.loc[:, 'CPU_USAGE']	every row, one column
df6.loc[:3]	rows 0, 1, 2, 3
df6.loc[2:5]	rows 2, 3, 4, 5
df6.loc[:3, 'CPU_USAGE':'num_threads']	rows 0–3; CPU_USAGE, cutime, lru, num_threads

Not like a list

```
[10, 20, 30, 40][:3]
# [10, 20, 30]
```

A Python list slice stops **before** the end. A .loc slice works with labels, so it stops **at** the end label.

05 · VISUALIZATION

Every pandas plot is drawn by matplotlib

```
import matplotlib.pyplot as plt

plt.plot(x, y) # line graph
plt.bar(x, heights) # bar chart
plt.pie(values) # pie chart
plt.show() # display it
```

matplotlib is not part of standard Python. Install it once with pip install matplotlib, or !pip install matplotlib in a notebook cell.

Same chart, two ways to ask

Chart	pyplot	pandas
Line	plt.plot	kind='line'
Bar	plt.bar	kind='bar'
Pie	plt.pie	kind='pie'
Box	plt.boxplot	kind='box'

pandas calls pyplot for you, so plt.title() and the other functions coming up work on pandas plots too.

05 · VISUALIZATION

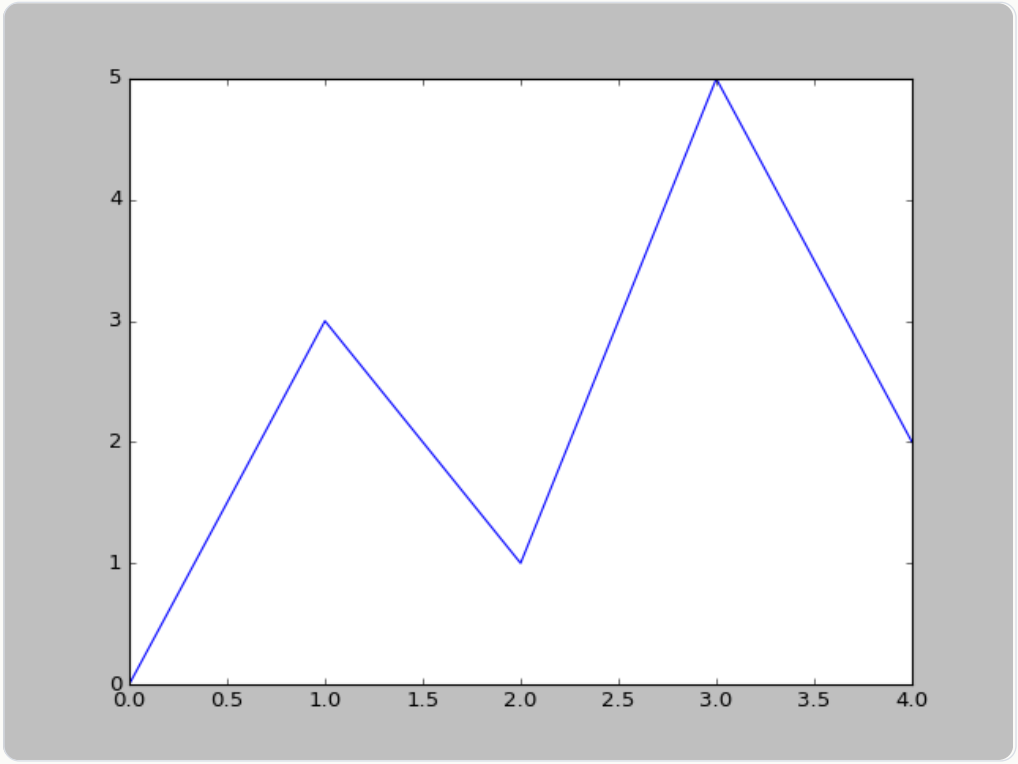
A line graph connects points in order

```
import matplotlib.pyplot as plt

x_coords = [0, 1, 2, 3, 4]
y_coords = [0, 3, 1, 5, 2]

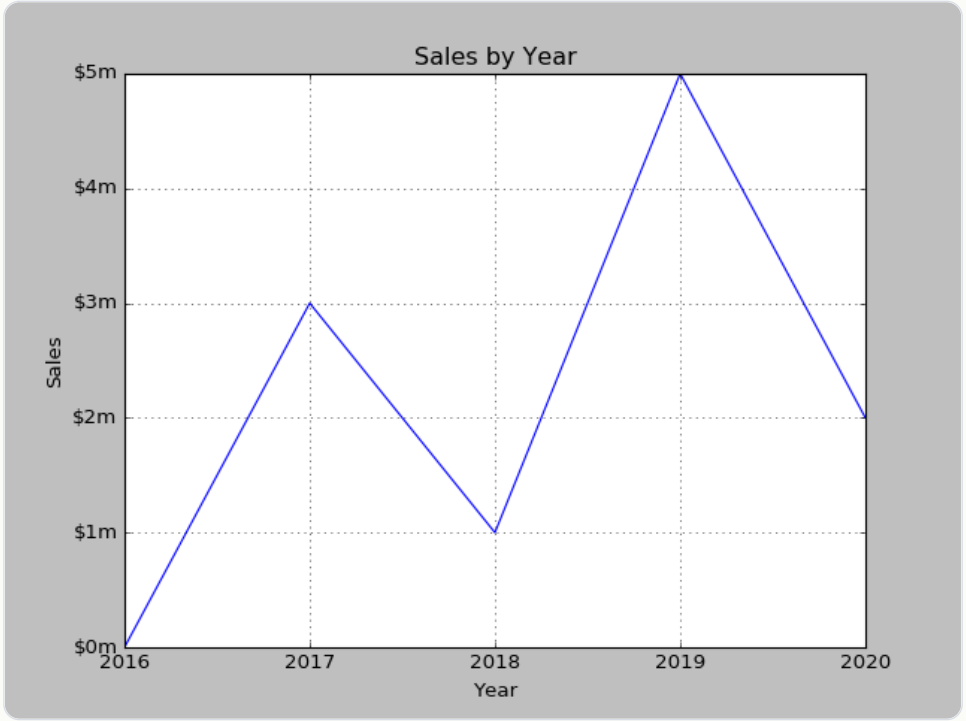
plt.plot(x_coords, y_coords)
plt.show()
```

Two lists of the same length: the **first** holds the x values, the **second** the y values. Point 1 is (0, 0), point 2 is (1, 3), and so on.



05 · VISUALIZATION

Label what the reader is looking at



Add this line	To get
plt.title('Sales by Year')	the title on top
plt.xlabel('Year')	the x-axis name
plt.ylabel('Sales')	the y-axis name
plt.xticks([0, 1, 2, 3, 4], ['2016', ..., '2020'])	years instead of 0–4
plt.yticks([0, 1, ... 5], ['$0m', ..., '$5m'])	dollar labels
plt.grid(True)	the dotted grid
plt.xlim(1, 100) · plt.ylim(10, 50)	set each axis's range (not used here)

05 · VISUALIZATION

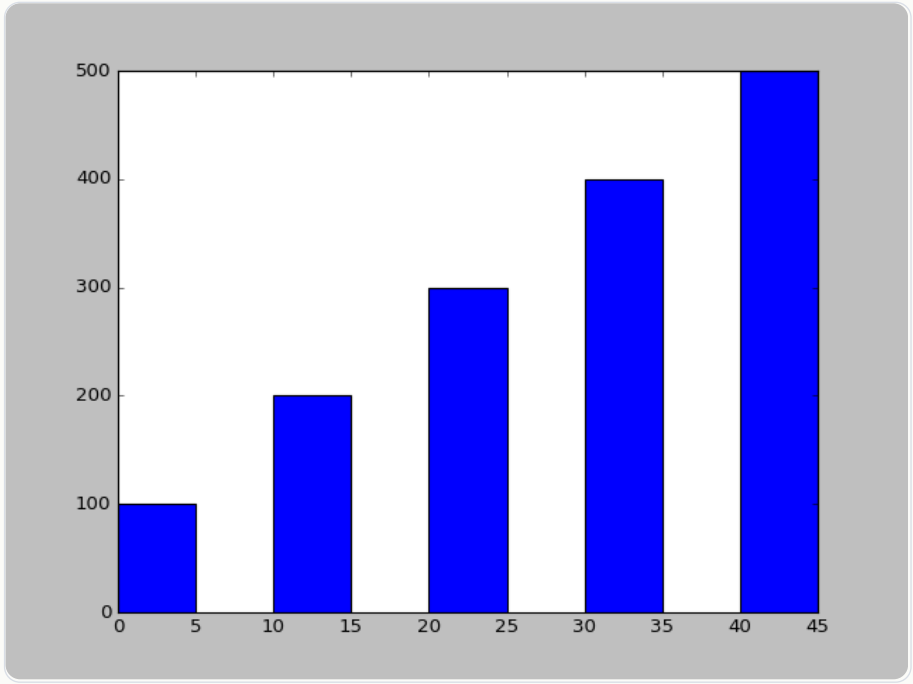
A bar chart compares amounts

```
left_edges = [0, 10, 20, 30, 40]
heights = [100, 200, 300, 400, 500]
plt.bar(left_edges, heights, 5, align='edge')
plt.show()
```

The third argument is the bar **width** (default 0.8). Bars are centered on their x values unless you pass align='edge', as here.

Color codes for color=('r', 'g', 'b', 'w', 'k')

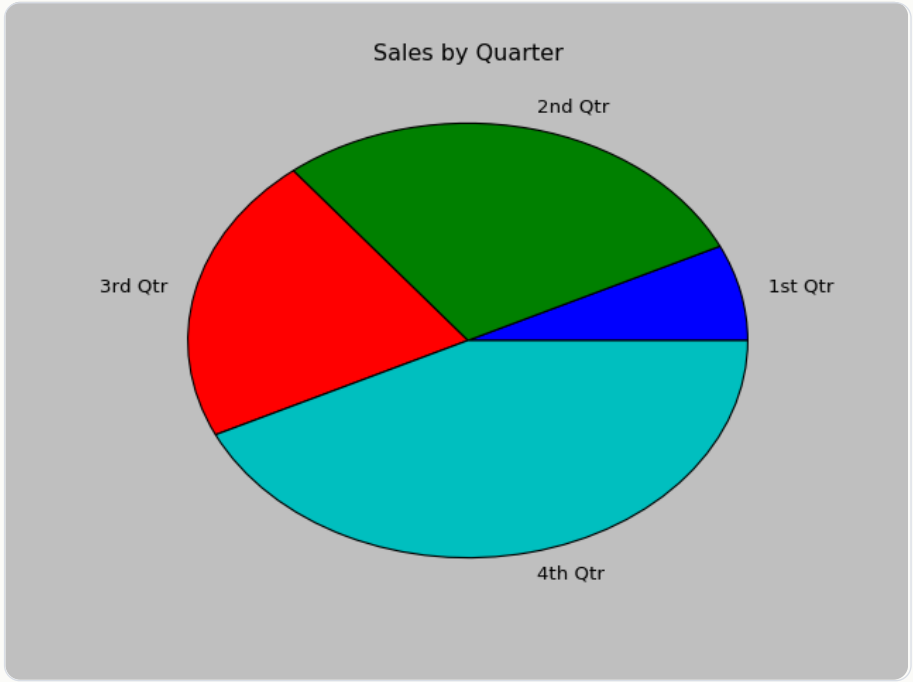
'b'	'g'	'r'	'c'	'm'	'y'	'k'	'w'
blue	green	red	cyan	magenta	yellow	black	white



05 · VISUALIZATION

A pie chart shows parts of a whole

```
sales = [100, 400, 300, 600]
slice_labels = ['1st Qtr', '2nd Qtr',
'3rd Qtr', '4th Qtr']
plt.pie(sales, labels=slice_labels)
plt.title('Sales by Quarter')
plt.show()
```



Each value becomes one slice, sized by its share of the total. The 4th quarter's 600 out of 1,400 fills **43%** of the pie.

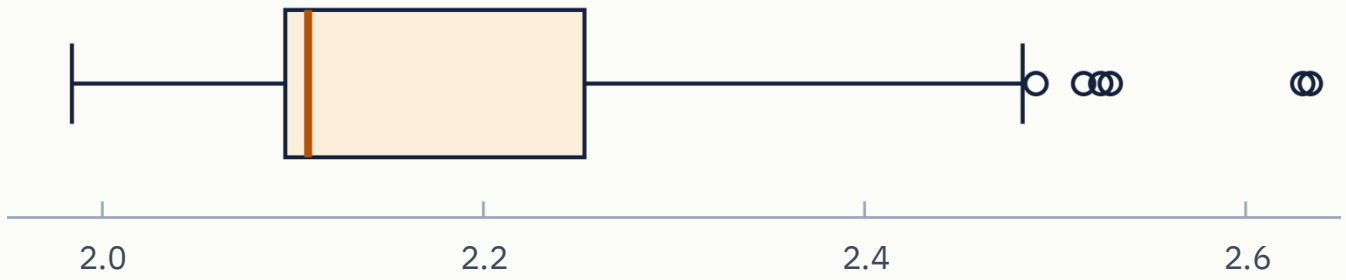
Choose slice colors with colors=('r', 'g', 'b', 'w', 'k'). Without it, current matplotlib uses blue, orange, green, red ...

05 · VISUALIZATION

A box plot summarizes a column in one picture

```
df_mystery['vsize'].plot(kind='line') # raw values, in row order
df_mystery['vsize'].plot(kind='box') # the summary below
seaborn.boxplot(x=df_mystery['vsize'], color='cyan')
```

vsize (virtual memory) of the 200 snapshots, in GB



Most snapshots sit near 2.1 GB. Circles are outliers: values more than 1.5 × the box width beyond it.

describe()	GB	In the plot
min	1.98	left whisker
25%	2.10	box, left edge
50% (median)	2.11	amber line
75%	2.25	box, right edge
max	2.63	farthest circle

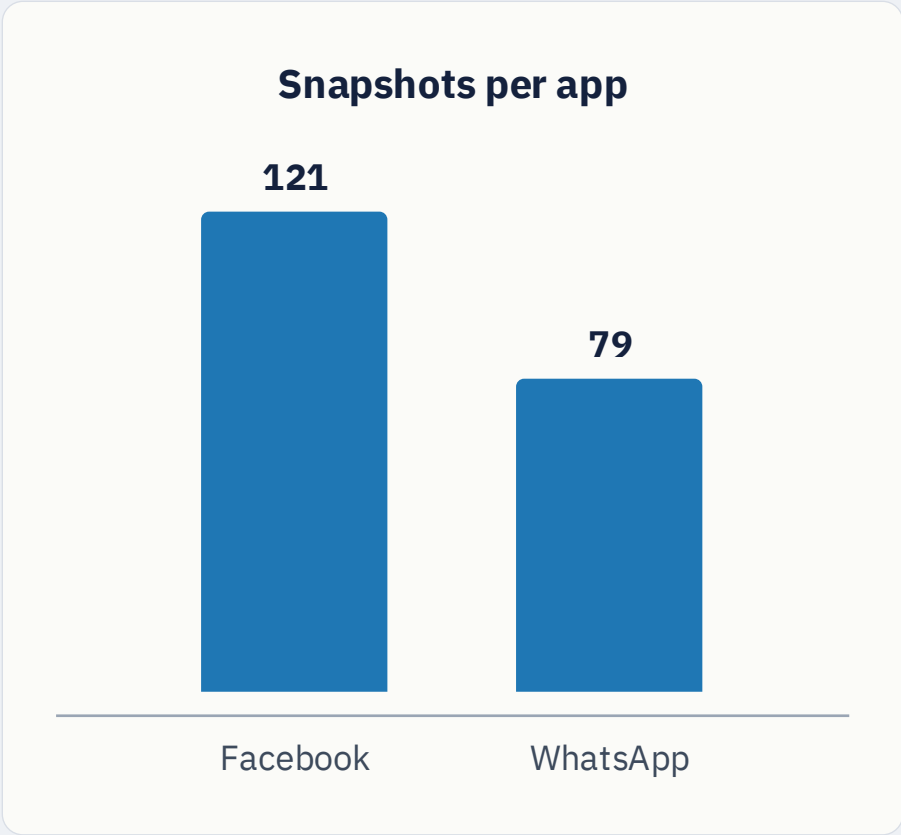
05 · VISUALIZATION

The same charts, straight from a DataFrame

```
counts = df_mystery['ApplicationName'].value_counts()
counts.plot(kind='bar')
plt.title('Snapshots per app')
plt.ylabel('Rows')
```

value_counts() counts the rows for each app. pandas uses the labels (Facebook, WhatsApp) for the bars, so there is no x list to build.

Try kind='pie' on counts, or df_mystery['CPU_USAGE'].plot(kind='hist', bins=20). One plot per cell.



YOUR TURN · NOTEBOOK SECTION 3.4

Four exercises on df_mystery

1 Spot the difference

What changes between each pair?

`df_mystery['CPU_USAGE']` vs `df_mystery[['CPU_USAGE']]`

`df_mystery.loc[1]` vs `df_mystery.loc[[1]]`

2 Cut out a block

Rows labeled 5 through 15, and only the columns from ApplicationName through priority. Both ends included.

3 Drop the junk column

The first column, Unnamed: 0, is just a record number. Make a new DataFrame without it. Hint: it is the first column, so one compact slice does it.

4 Fill in a default

Every cminflt value in df6 is NaN (missing). Zero is a safe default: set the whole column to 0 with one .loc assignment.

RECAP

Six things to remember

Series and DataFrame

A labeled column, and a table of them.

Labels, not positions

s[3] finds the value labeled 3.

[] for columns, .loc[] for rows

And .loc[] to change values.

.loc slices include the end

df.loc[:3] is four rows.

= is not a copy

Use .copy() to experiment safely.

Explore, then plot

shape, info(), describe(), then plot(kind='box').

Keep handy: the lesson page with its indexing summary table, and the pandas cheat sheet. Next session: analytics with filtering, sorting and grouping.